Data Structures and Algorithms

Lecture 8

AVL and Red black Tree

Dept. of ICT, Faculty of Technology- UoR, 2026

Balanced search trees

- **AVL Trees**
- 2-3 trees
- 2-3-4 trees
- B-trees
- **Red-Black trees**

AVL Tree

- AVL tree is a self balanced binary search tree. That means, an AVL tree is also a binary search tree but it is a balanced tree.

- The balance factor of every node is no more than 1.

- The balance factor is often stored at each tree's node. However, a node's balance factor can be also computed from the heights of its subtrees.

AVL Trees (1962)

- Named after 2 Russian mathematicians
- Georgii Adelson-Velsky (1922 - ?)
- Evgenii Mikhailovich Landis (1921-1997)



Balance factor of AVL tree

- An AVL tree is a balanced binary search tree. Balance factor of every node is either -1, 0 or 1.

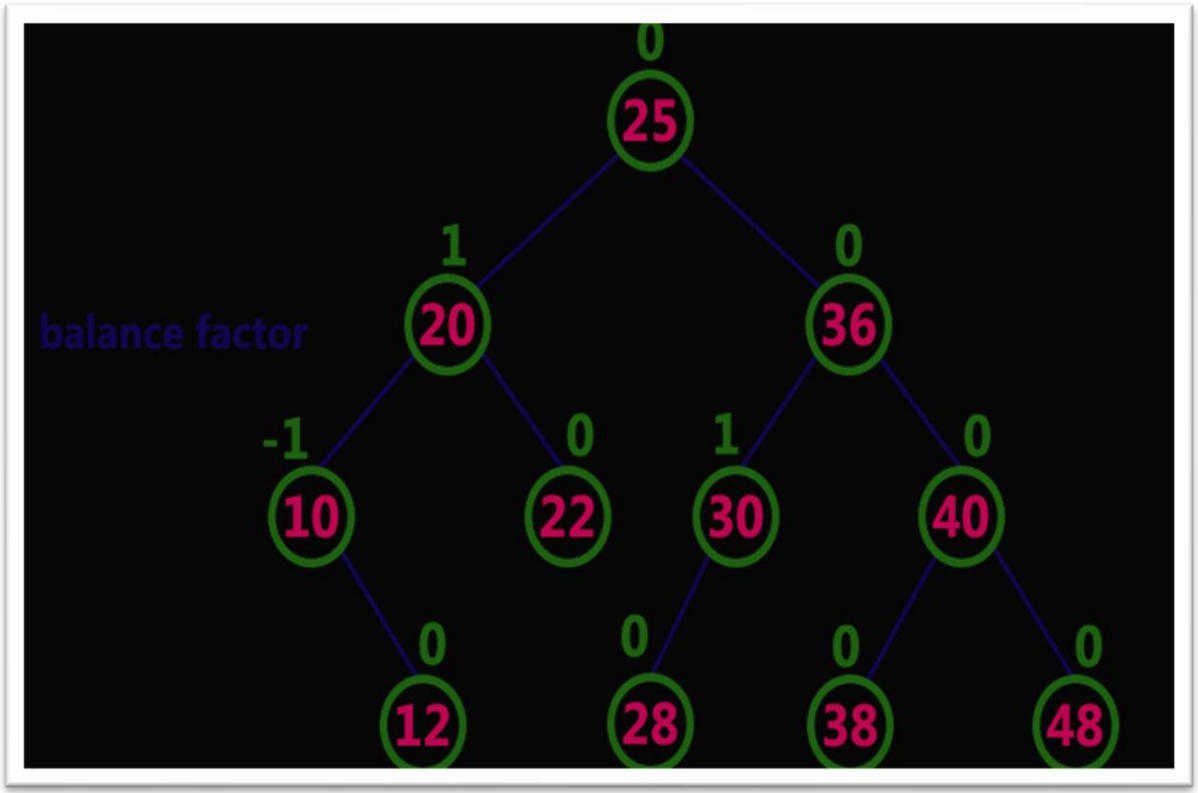
- A binary tree is said to be balanced, if the difference between the heights of left and right subtrees of every node in the tree is either -1, 0 or 1.

  (if it is other than these three values then the tree in not balance or not an AVL tree)

- In other words, a binary tree is said to be balanced if for every node, height of its children differ by at most one.

Balance factor = heightOfLeftSubtree - heightOfRightSubtree

Balance factor Calculation

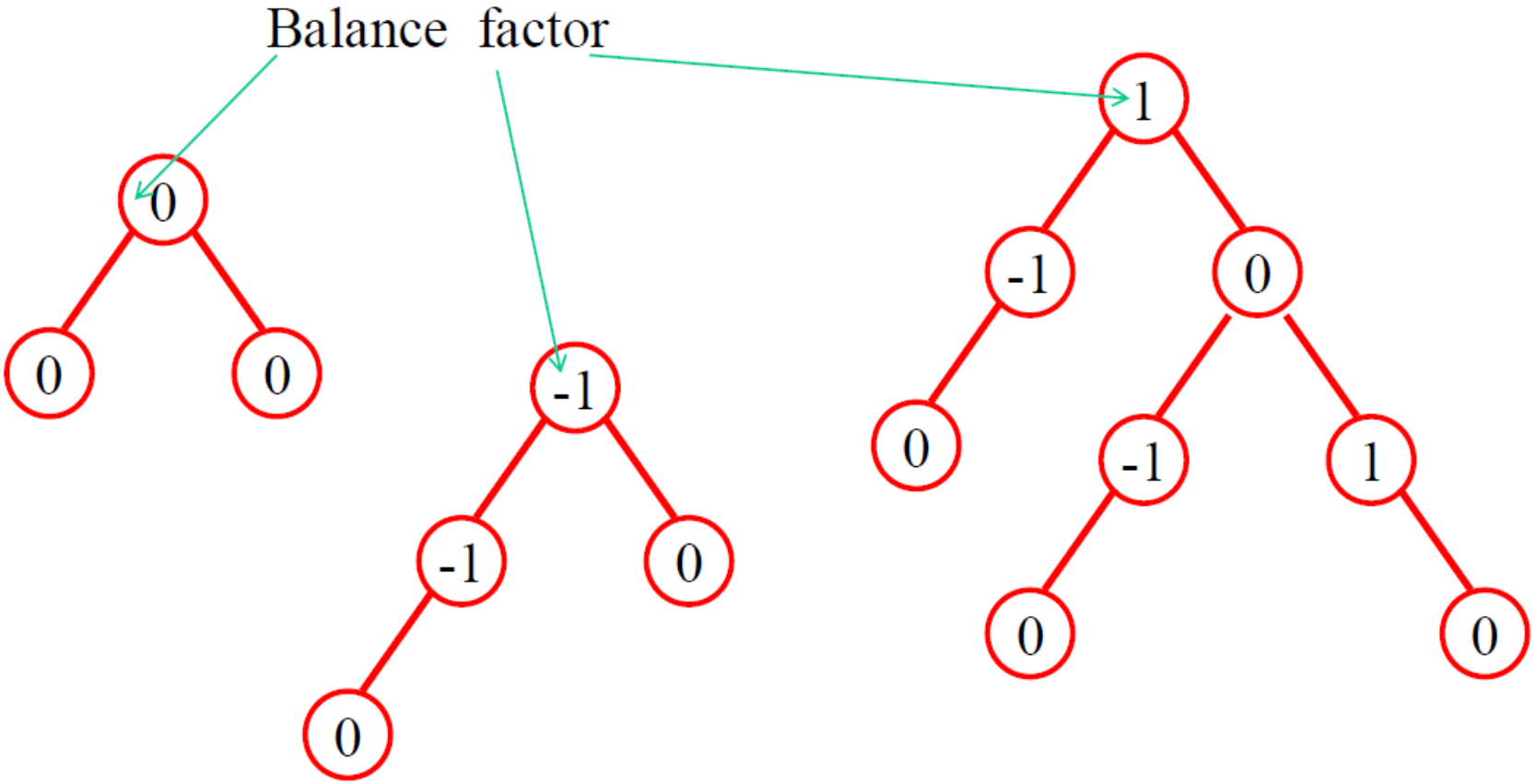


Above tree is a binary search tree and every node is satisfying balance factor condition. This tree is said to be an AVL tree.

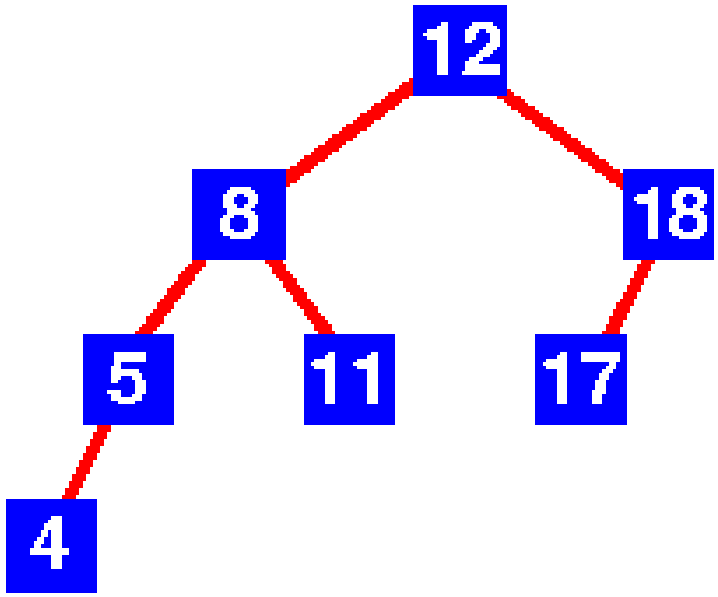
Balance factor = heightOfLeftSubtree - heightOfRightSubtree

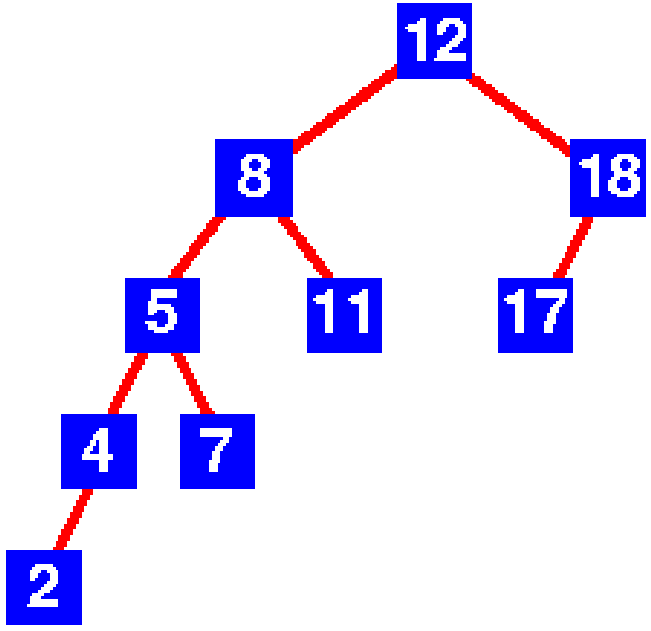
If every node in the tree is either -1, 0 or 1, then this type of tree called AVL tree

Examples of AVL Trees



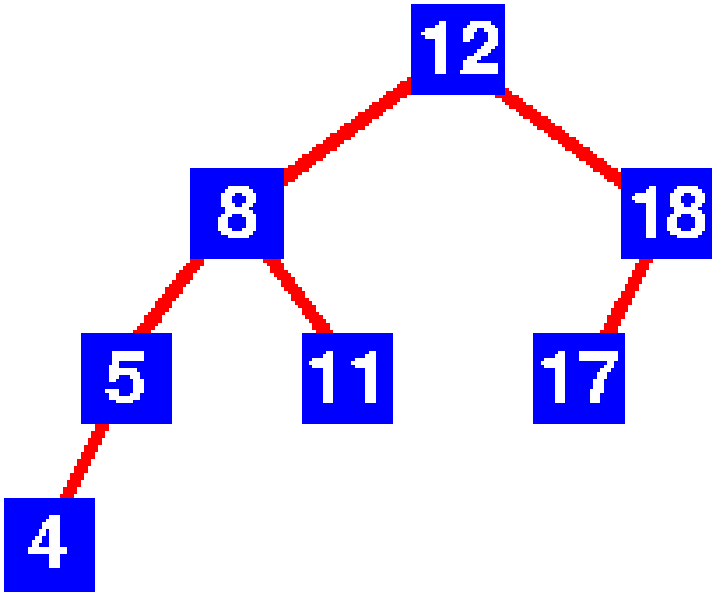
AVL Tree





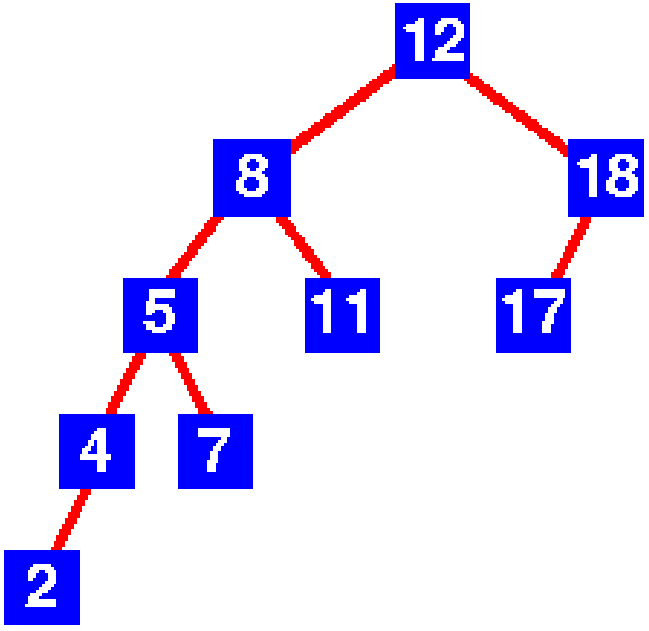
Every AVL Tree is a binary search tree but all the Binary Search Trees need not to be AVL trees.

AVL Tree



Each left sub-tree has a height 1 greater than each right sub-tree.

This is an AVL tree.



Sub-tree with root 8 has height 3 and sub-tree with root 18 has height 1.

This is not an AVL tree.

Every AVL Tree is a binary search tree but all the Binary Search Trees need not to be AVL trees.

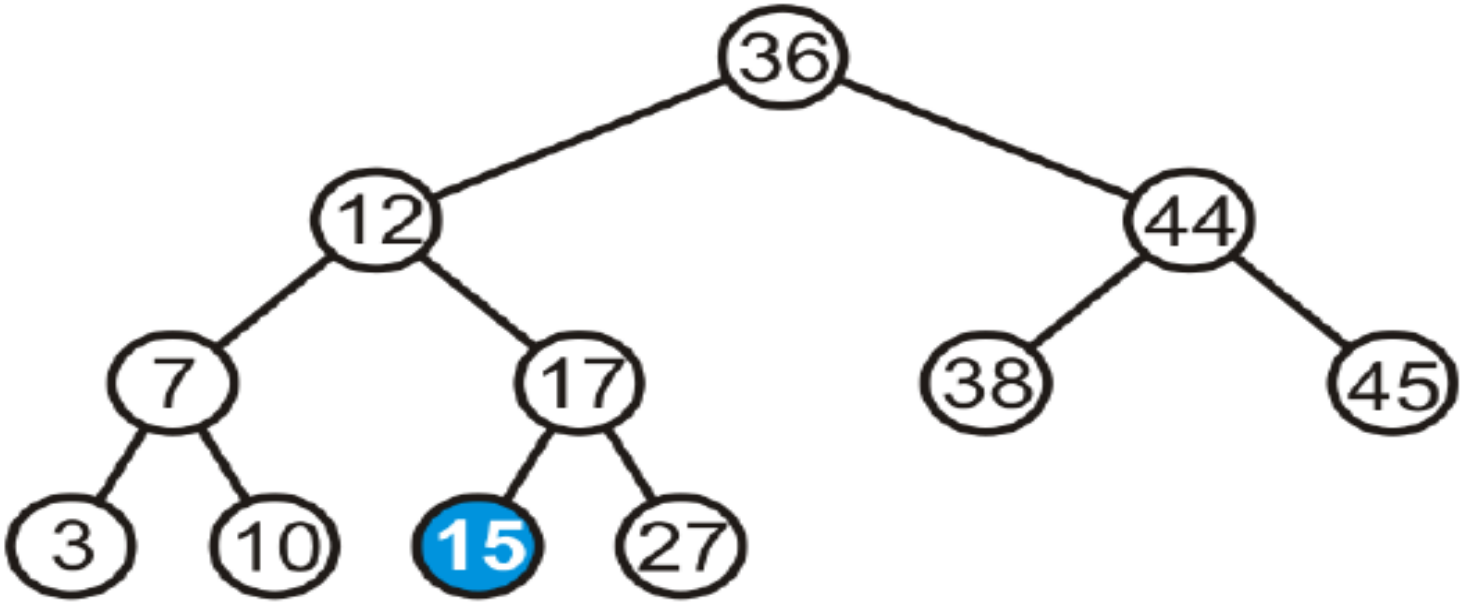
Maintaining Balance

To maintain AVL balance, observe that:

- Inserting a node can increase the height of a tree by at most 1
- Removing a node can decrease the height of a tree by at most 1

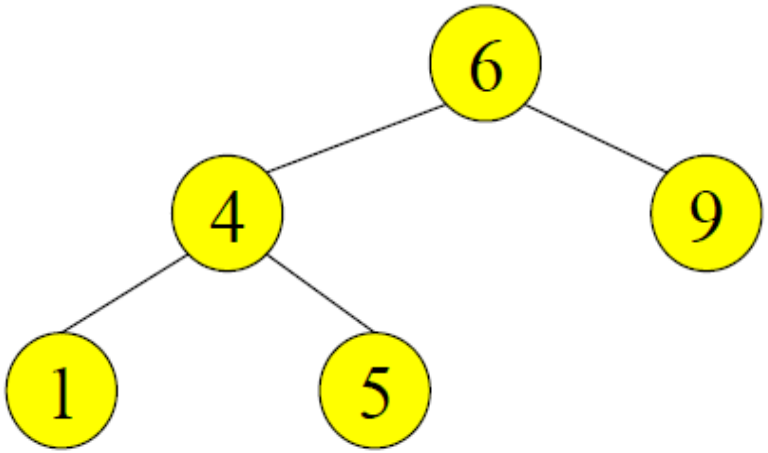
Maintaining Balance

- Consider inserting 15 into this tree
- In this case, the heights of none of the trees change.

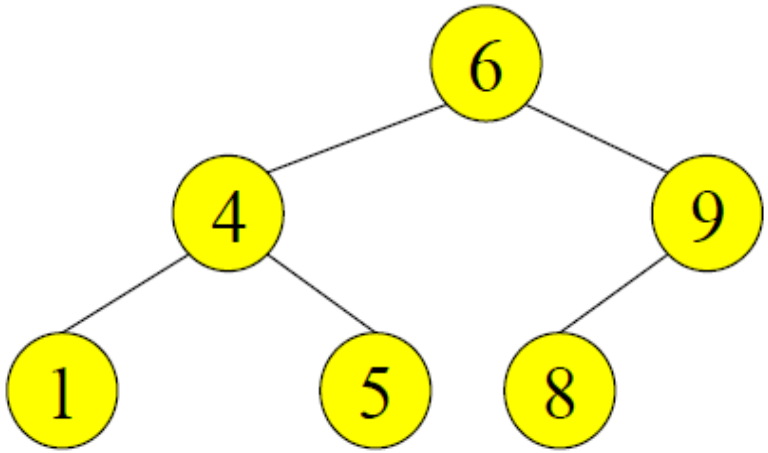


Are these trees AVL or not?

Tree A



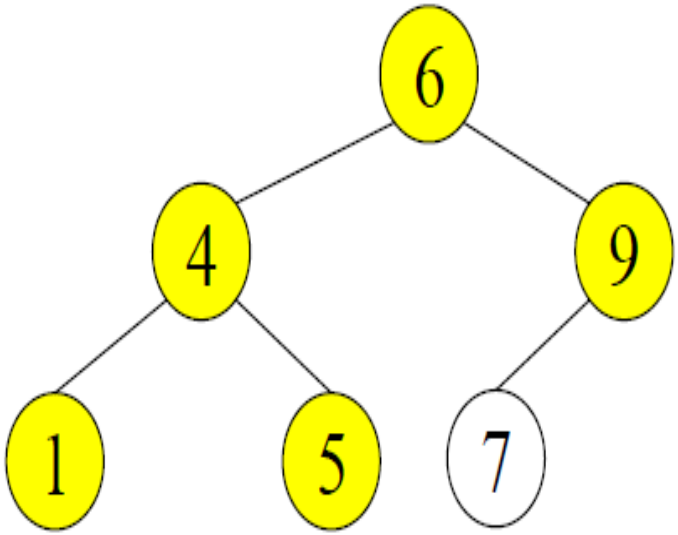
Tree B



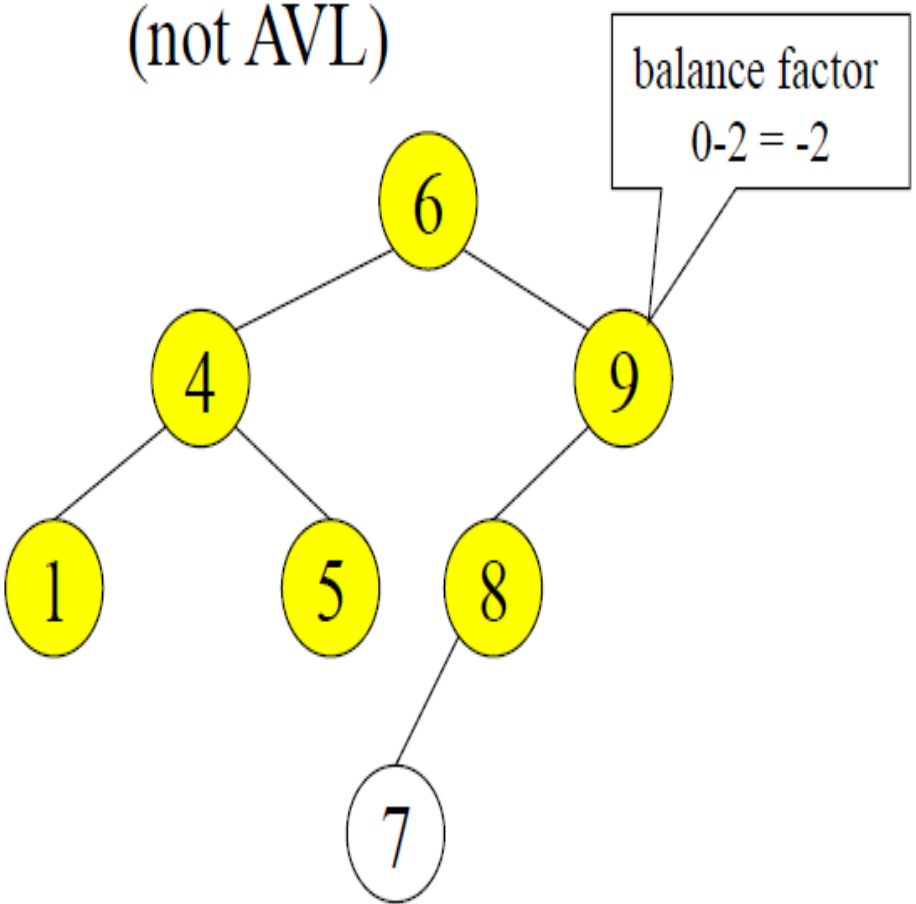
Insert 7 to both of above trees. Are these trees now AVL or not?

Trees after Inserting 7

Tree A
(AVL)



Tree B
(not AVL)



Insertion and Rotation in AVL Trees

Insert operation may cause balance factor to become 2 or –2 for some nodes.

- only nodes on the path from insertion point to root node have possibly changed in height.
- So after the Insertion, go back up to the root node by node, updating heights.
- If a new balance factor is 2 or –2, adjust tree by rotation around the node

AVL Tree Data Structure

- Extension of binary search tree.
- Maintain AVL property using rotations
- Rotations occur when tree becomes unbalanced from insertion or deletion
- At each node, keep track of height of sub-tree rooted at node (for balance factor computations)

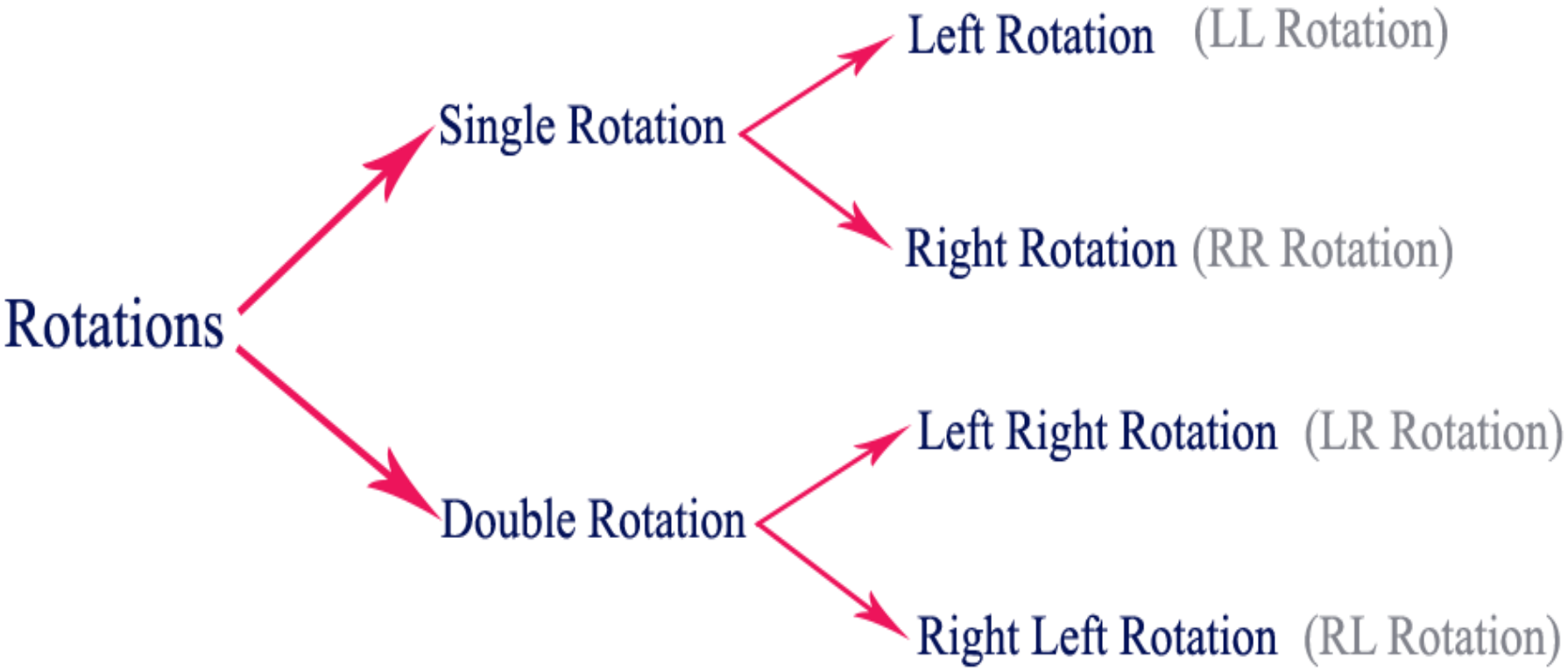
AVL Tree Rotations

- AVL tree, after performing every operation like insertion and deletion we need to check the balance factor of every node in the tree.
- If every node satisfies the balance factor condition then we conclude the operation otherwise we must make it balanced.
- We use rotation operations to make the tree balanced whenever the tree is becoming imbalanced due to any operation.

Rotation is the process of moving the nodes to either left or right to make tree balanced.

AVL Tree Rotation operation

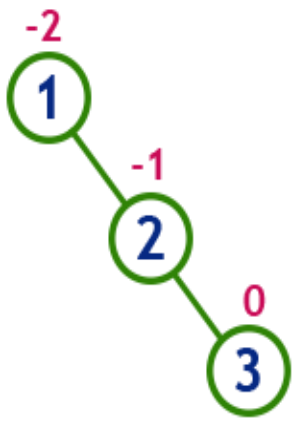
four rotations and they are classified into **two** types



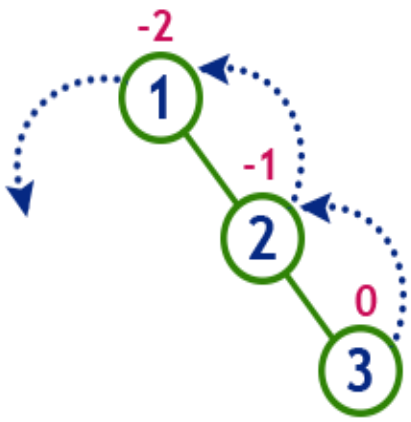
Single Left Rotation (LL Rotation)

In LL Rotation every node moves one position to left from the current position. To understand LL Rotation, let us consider following insertion operations into an AVL Tree.

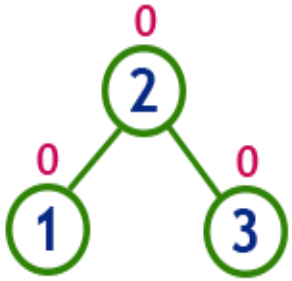
insert 1, 2 and 3



Tree is imbalanced



To make balanced we use LL Rotation which moves nodes one position to left

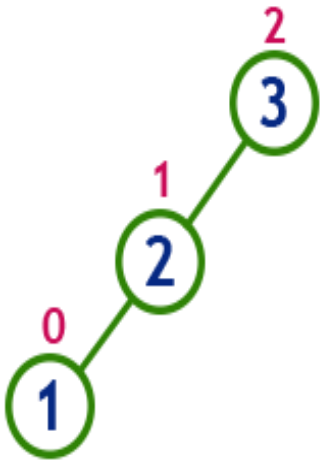


After LL Rotation Tree is Balanced

Single Right Rotation (RR Rotation)

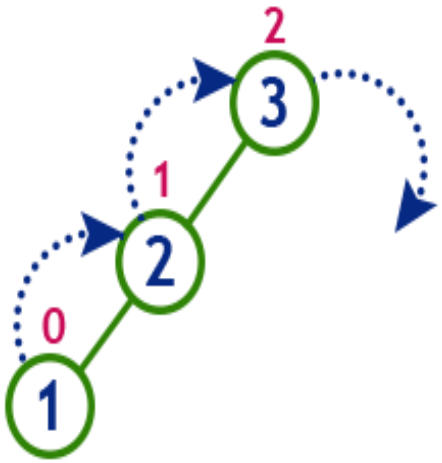
In RR Rotation every node moves one position to right from the current position. To understand RR Rotation, let us consider following insertion operations into an AVL Tree.

insert 3, 2 and 1

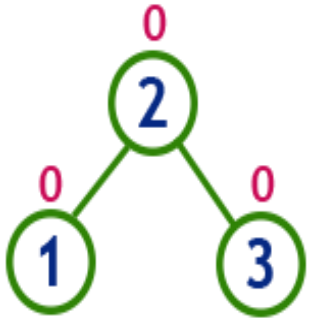


Tree is imbalanced

because node 3 has balance factor 2



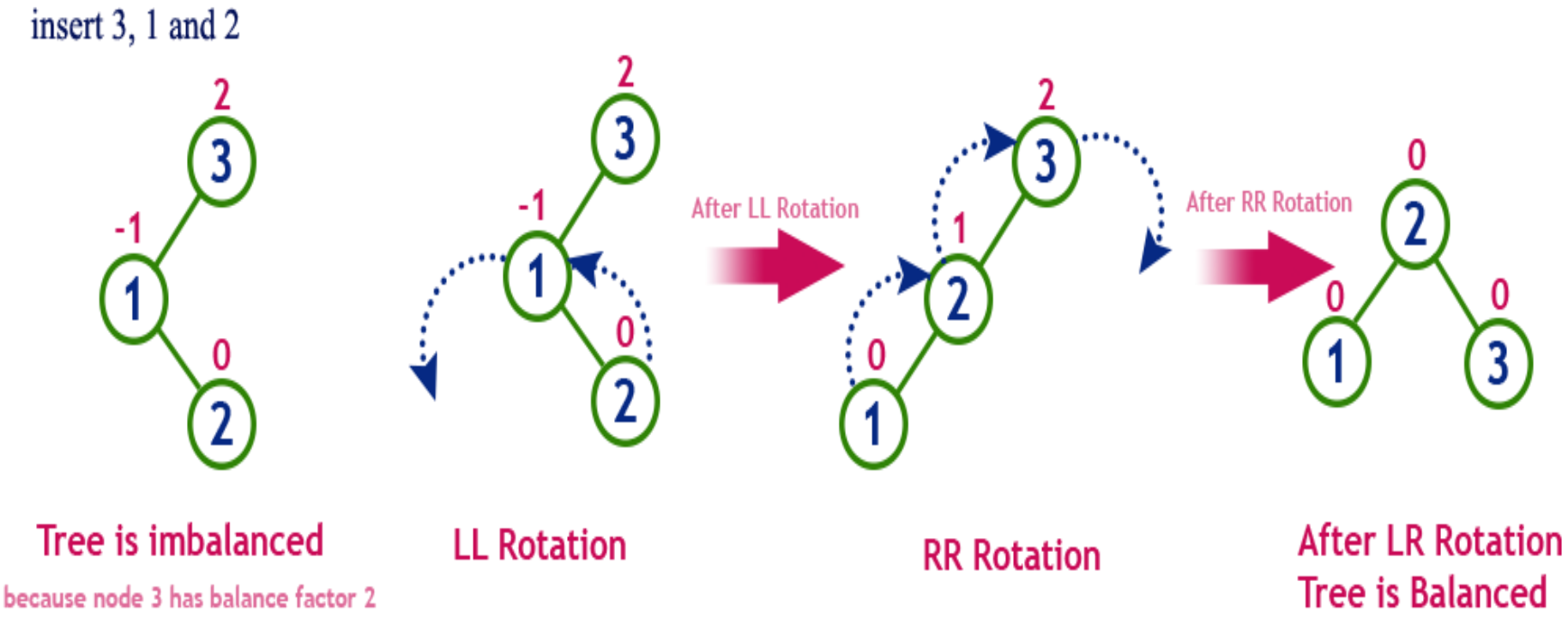
To make balanced we use RR Rotation which moves nodes one position to right



After RR Rotation Tree is Balanced

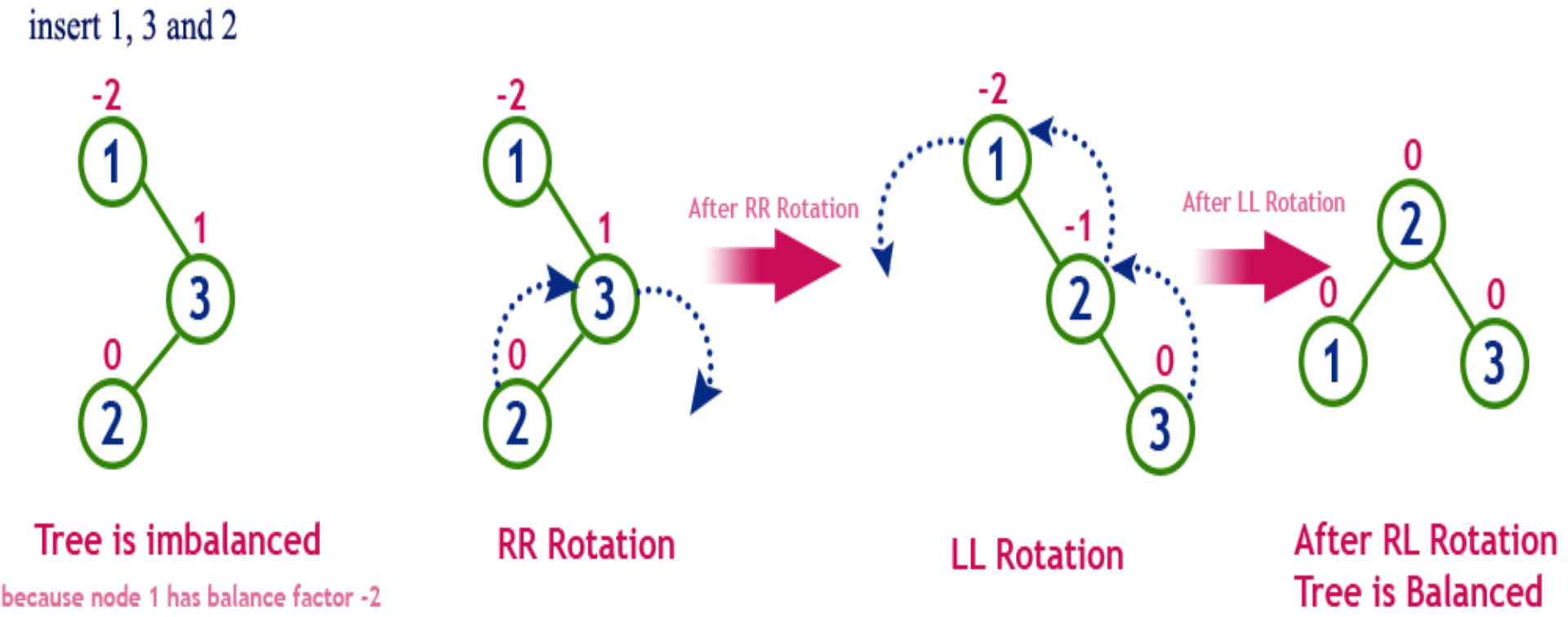
Left Right Rotation (LR Rotation)

The LR Rotation is combination of single left rotation followed by single right rotation. In LR Roration, first every node moves one position to left then one position to right from the current position. To understand LR Rotation, let us consider following insertion operations into an AVL Tree.



Right Left Rotation (RL Rotation)

The RL Rotation is combination of single right rotation followed by single left rotation. In RL Rotation, first every node moves one position to right then one position to left from the current position. To understand RL Rotation, let us consider following insertion operations into an AVL Tree.



Operations on an AVL Tree

1. Search
2. Insertion
3. Deletion

Search Operation in AVL Tree

The search operation is performed similar to Binary search tree search operation.

Step 1: Read the search element from the user

Step 2: Compare, the search element with the value of root node in the tree.

Step 3: If both are matching, then display "Given node found!!!" and terminate the function

Step 4: If both are not matching, then check whether search element is smaller or larger than that node value.

Step 5: If search element is smaller, then continue the search process in left subtree.

Step 6: If search element is larger, then continue the search process in right subtree.

Step 7: Repeat the same until we found exact element or we completed with a leaf node

Step 8: If we reach to the node with search value, then display "Element is found" and terminate the function.

Step 9: If we reach to a leaf node and it is also not matching, then display "Element not found" and terminate the function.

Insertion Operation in AVL Tree

In an AVL tree, the insertion operation is performed with O(log n) time complexity. In AVL Tree, new node is always inserted as a leaf node. The insertion operation is performed as follows

Step 1: Insert the new element into the tree using Binary Search Tree insertion logic.

Step 2: After insertion, check the Balance Factor of every node.

Step 3: If the Balance Factor of every node is 0 or 1 or -1 then go for next operation.

Step 4: If the Balance Factor of any node is other than 0 or 1 or -1 then tree is said to be imbalanced. Then perform the suitable Rotation to make it balanced. And go for next operation.

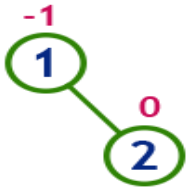
Construct an AVL Tree by inserting numbers from 1 to 8

insert 1



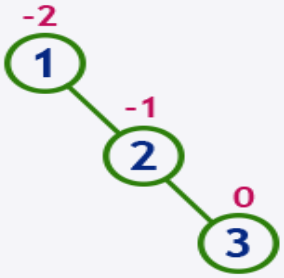
Tree is balanced

insert 2

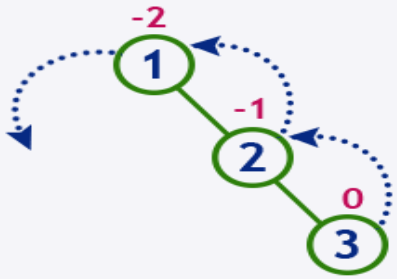


Tree is balanced

insert 3

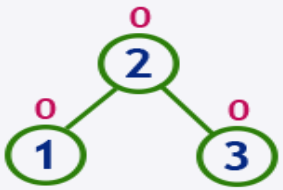


Tree is imbalanced



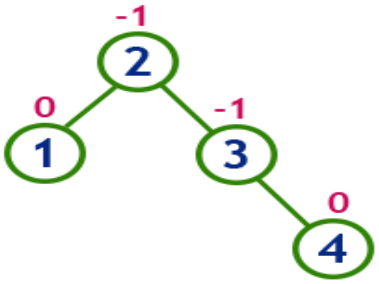
LL Rotation

After LL Rotation



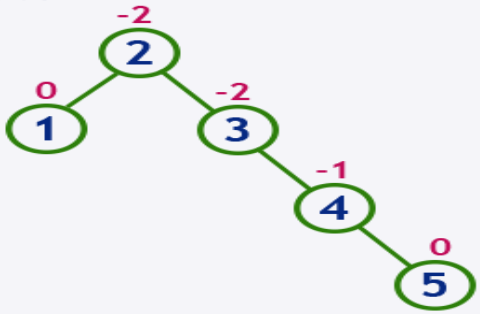
Tree is balanced

insert 4

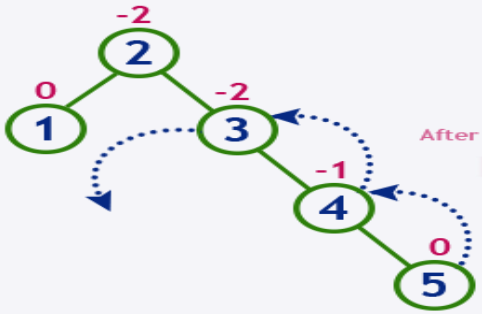


Tree is balanced

insert 5

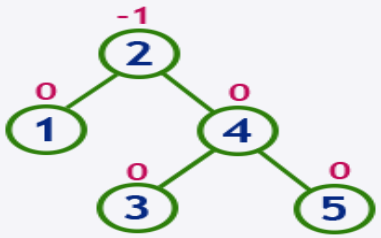


Tree is imbalanced



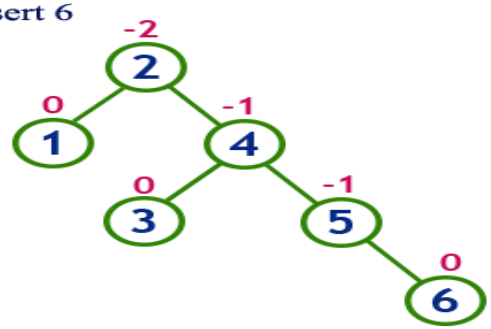
LL Rotation at 3

After LL Rotation at 3

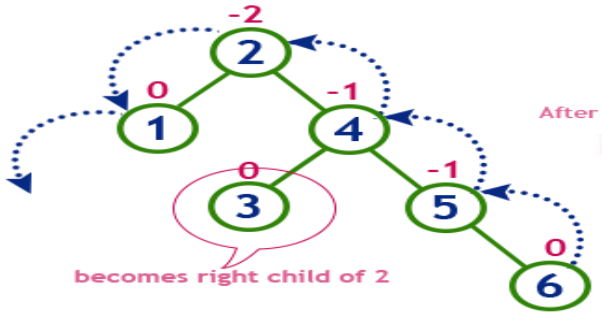


Tree is balanced

insert 6

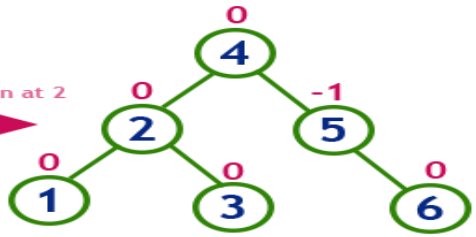


Tree is imbalanced



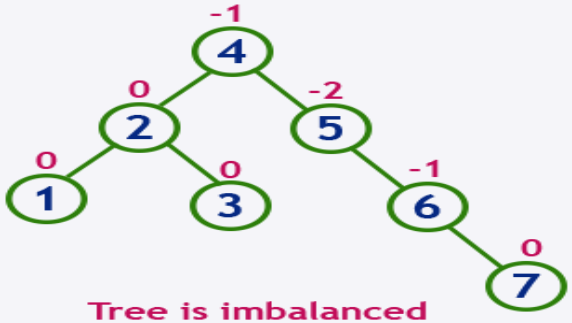
LL Rotation at 2

After LL Rotation at 2

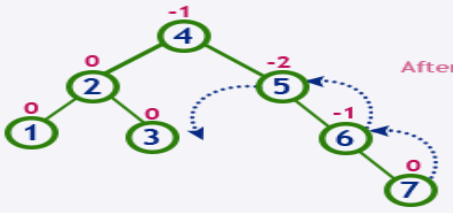


Tree is balanced

insert 7

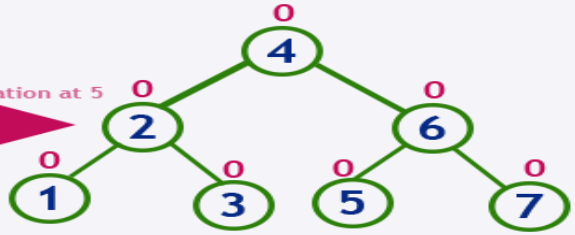


Tree is imbalanced



LL Rotation at 5

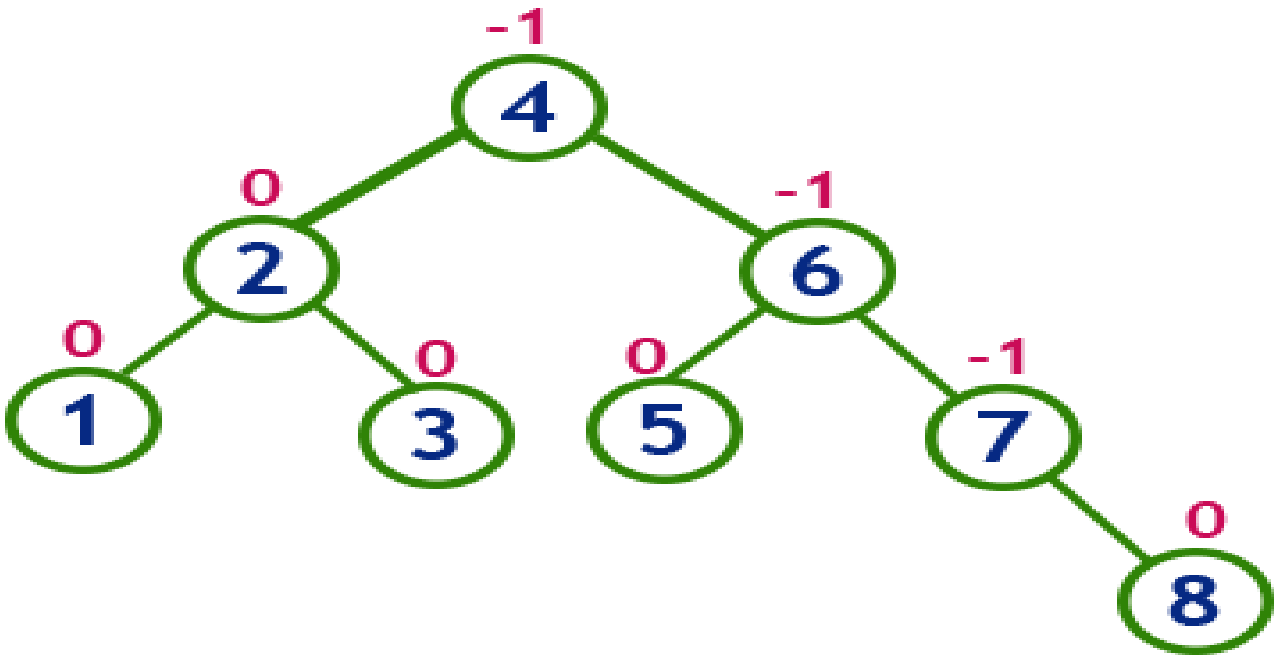
After LL Rotation at 5



Tree is balanced

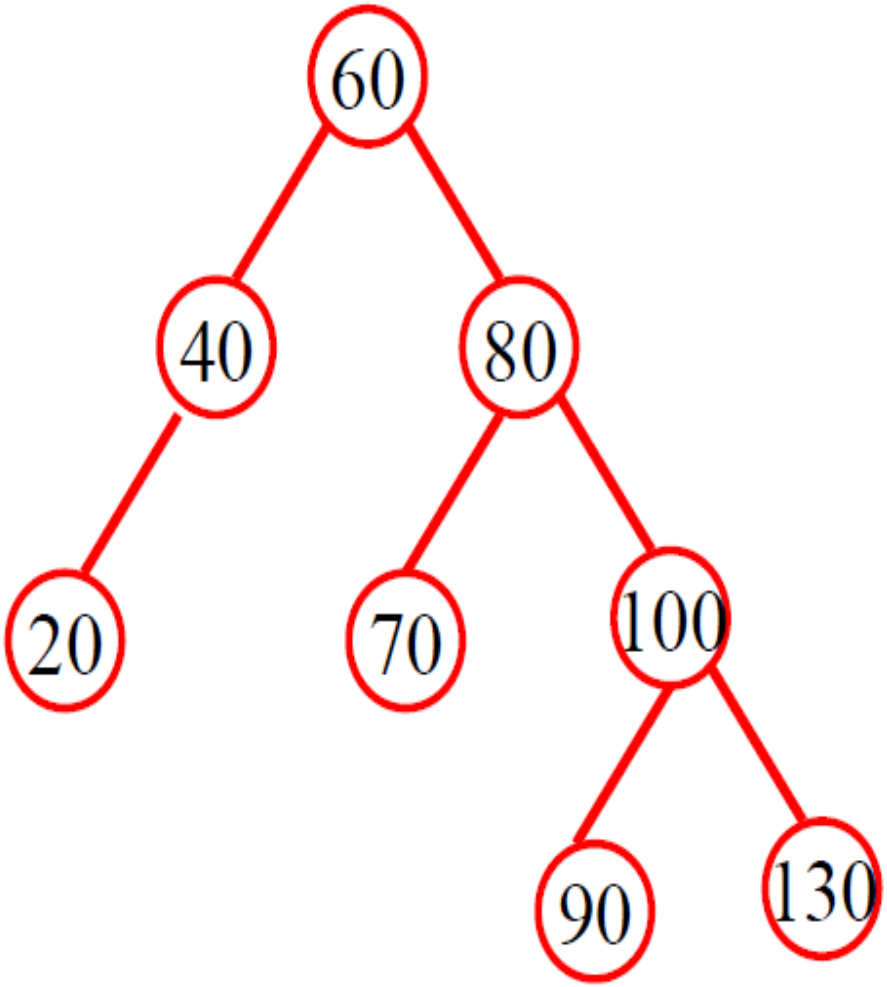
Construct an AVL Tree by inserting numbers from 1 to 8

insert 8

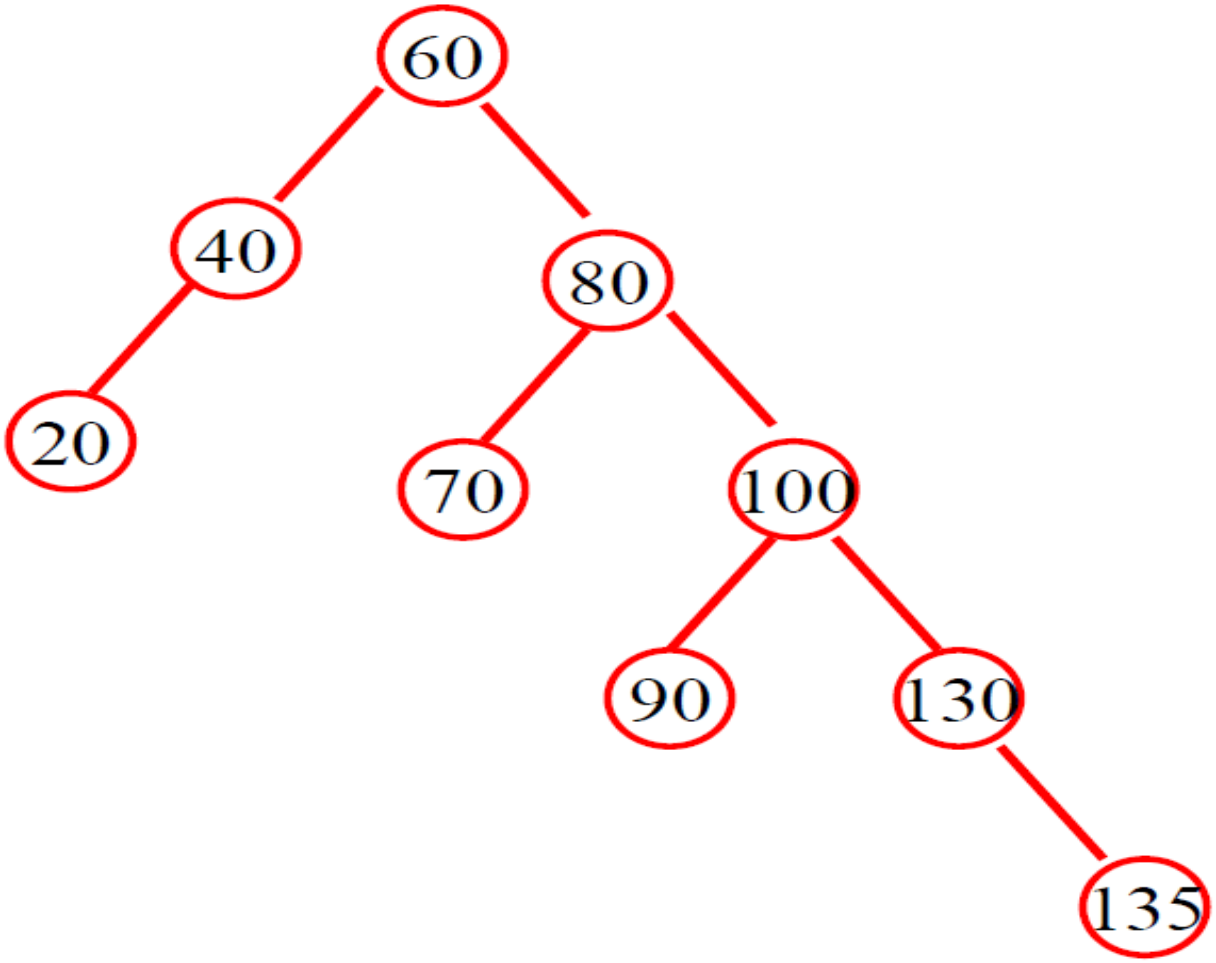


Tree is balanced

Consider the following tree. Insert 135 now.

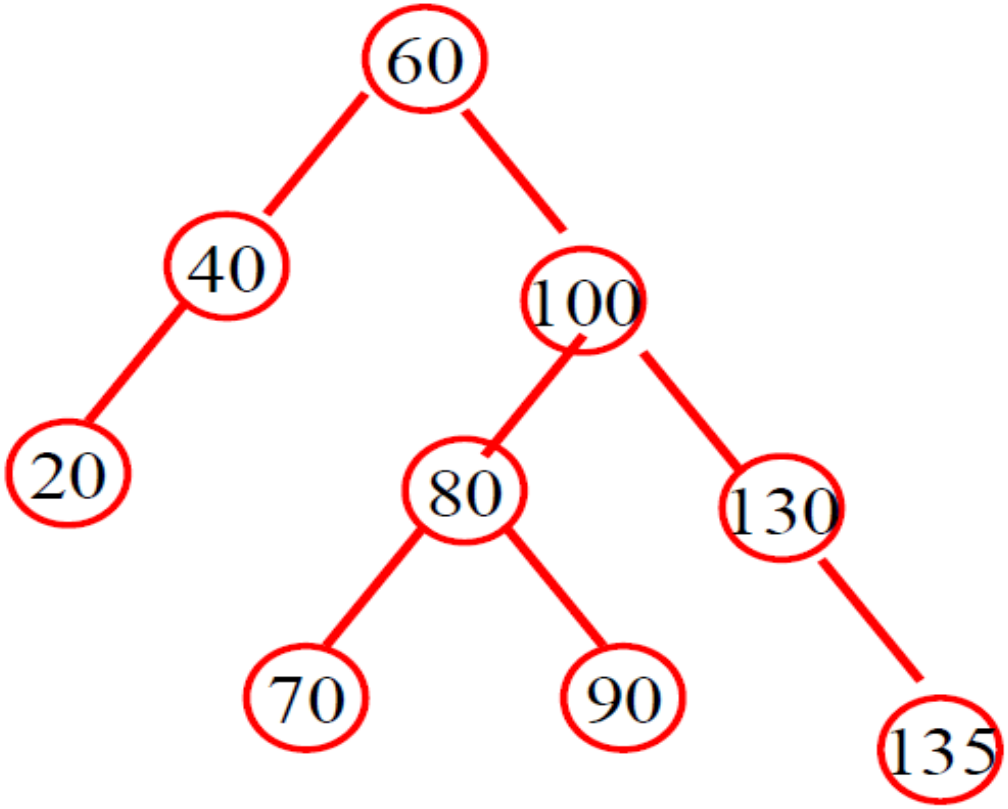


Resultant tree after inserting



Is this a balance tree or not?

Resultant tree after rotating



This is a balance tree

14 , 17, 11 ,7 ,53, 4, 13,12,8 ,60, 19, 16, 20

Deletion operation in AVL tree

- In an AVL Tree, the deletion operation is similar to deletion operation in BST.
- But after every deletion operation we need to check with the Balance Factor condition.
- If the tree is balanced after deletion then go for next operation otherwise perform the suitable rotation to make the tree Balanced.

Red black tree

Red black tree

Red - Black Tree is another variant of Binary Search Tree in which every node is colored either RED or BLACK.

Red black tree

A red black tree “colours” each node within a tree either red or black

- In AVL trees, balancing restricts the difference in heights to at most one
- For red-black trees, we have a different set of rules related to the colors of the nodes

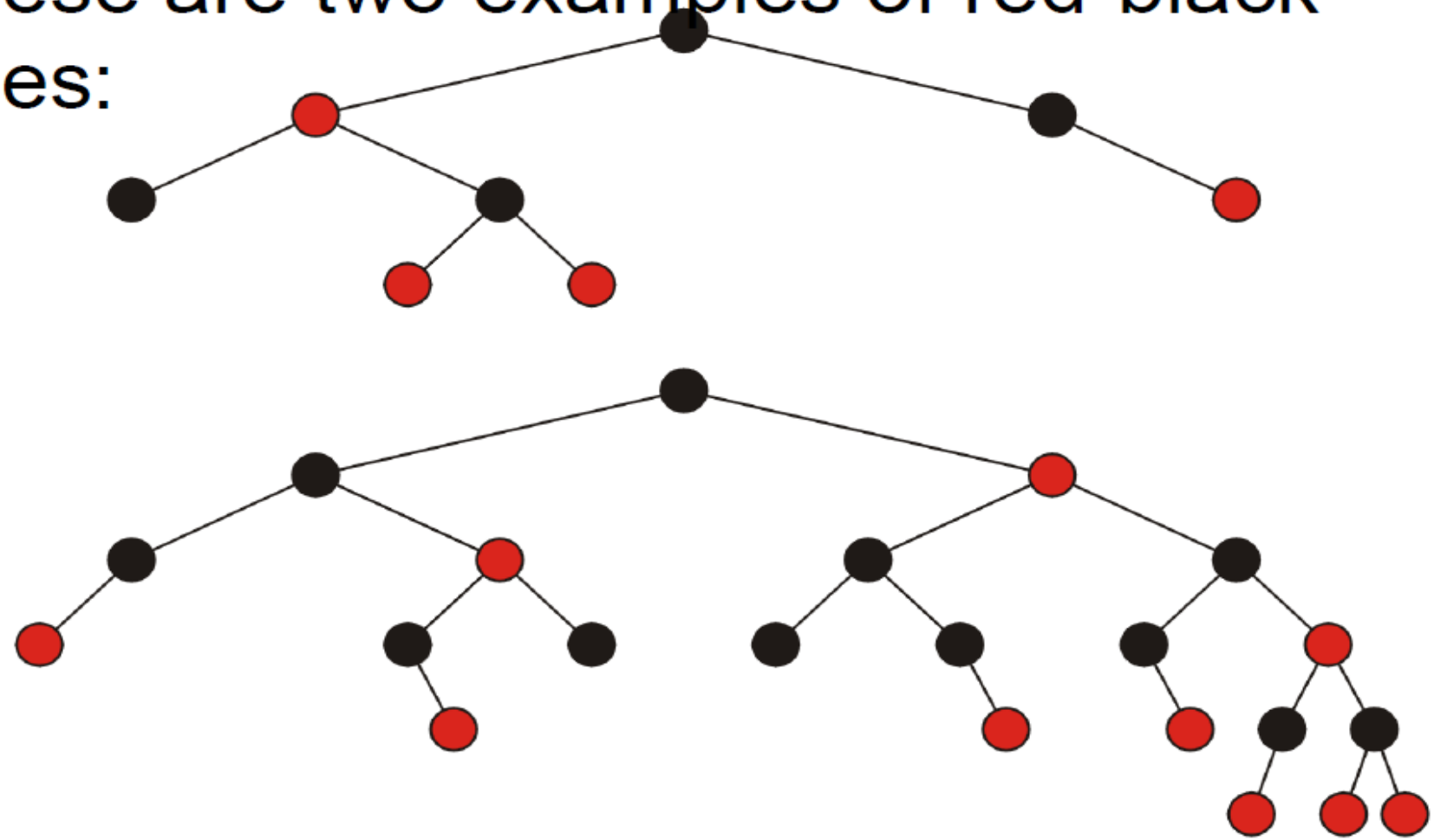
Red black tree

Red Black Tree the color of a node is decided based on the Red Black Tree properties. Every Red Black Tree has the following properties..

- **Property #1:** Red - Black Tree must be a Binary Search Tree.
- **Property #2:** The ROOT node must colored BLACK.
- **Property #3:** The children of Red colored node must colored BLACK. (There should not be two consecutive RED nodes).
- **Property #4:** In all the paths of the tree there must be same number of BLACK colored nodes.
- **Property #5:** Every new node must inserted with RED color.
- **Property #6:** Every leaf (e.i. NULL node) must colored BLACK.

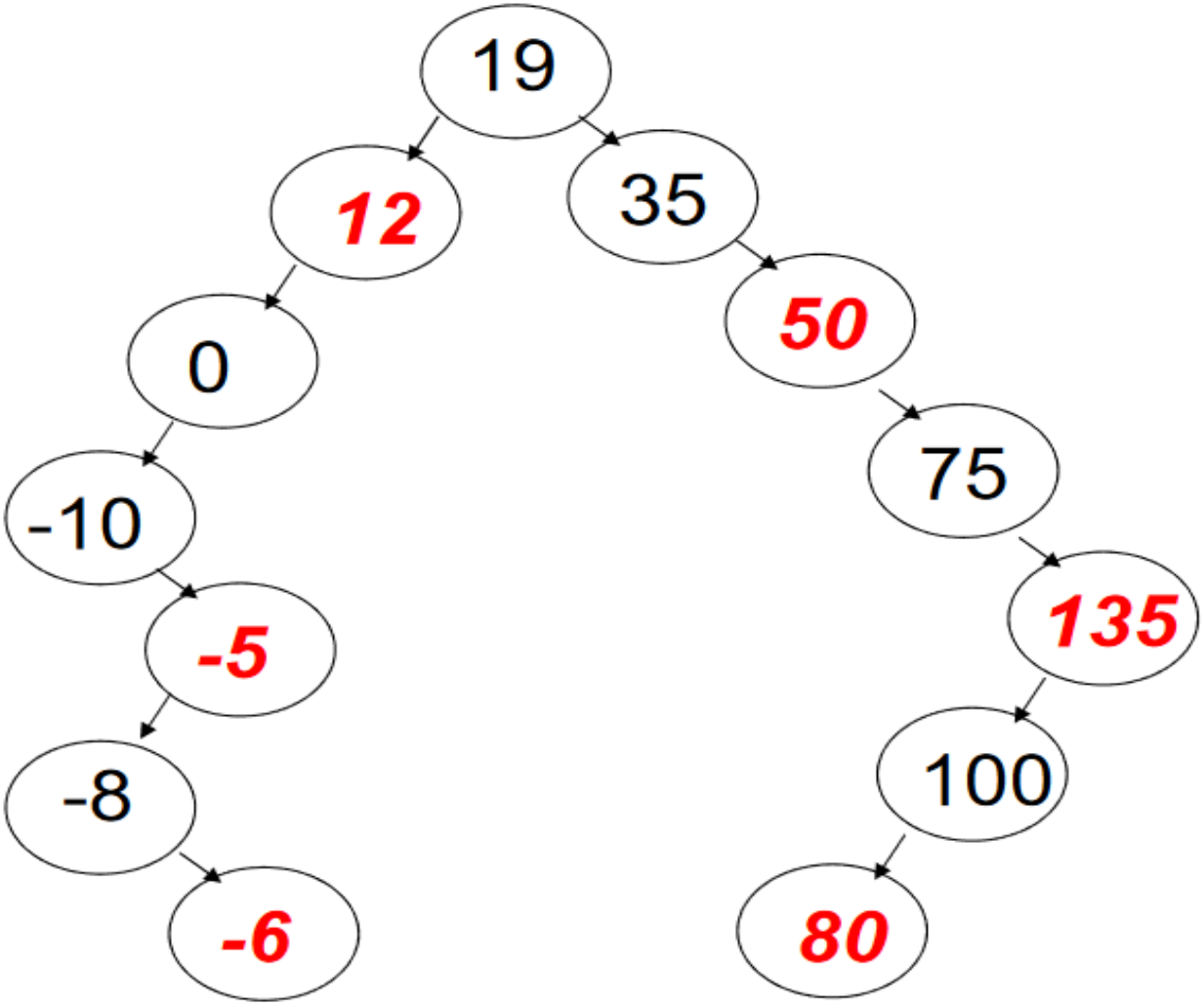
Red-Black Trees

These are two examples of red-black trees:



The above tree is a Red Black tree and every node is satisfying all the properties of Red Black Tree.

Red Black Tree?



Insertion into RED BLACK Tree:

- Red Black Tree, every new node must be inserted with color RED.
- The insertion operation in Red Black Tree is similar to insertion operation in Binary Search Tree. But it is inserted with a color property.
- After every insertion operation, we need to check all the properties of Red Black Tree.
- If all the properties are satisfied then we go to next operation otherwise we need to perform following operation to make it Red Black Tree.

  - Recolor
  - Rotation followed by Recolor

Insertion operation in Red Black tree

Step 1: Check whether tree is Empty.

Step 2: If tree is Empty then insert the newNode as Root node with color Black and exit from the operation.

Step 3: If tree is not Empty then insert the newNode as a leaf node with Red color.

Step 4: If the parent of newNode is Black then exit from the operation.

Step 5: If the parent of newNode is Red then check the color of parent node's sibling of newNode.

Step 6: If it is Black or NULL node then make a suitable Rotation and Recolor it.

Step 7: If it is Red colored node then perform Recolor and Recheck it. Repeat the same until tree becomes Red Black Tree.

END